

I. INTRODUCTION TO PANDAS: THE DATA ENGINE

While NumPy handles the raw numerical arrays, **Pandas** provides high-level data structures designed for practical data analysis in Python. It is built on top of NumPy, meaning it inherits NumPy's speed but adds the flexibility of labeled axes (rows and columns), similar to a SQL table or an Excel spreadsheet.

1.1. Core Data Structures

- **Series:** A 1-dimensional labeled array capable of holding any data type (integers, strings, floating-point numbers, Python objects, etc.).
- **DataFrame:** A 2-dimensional labeled data structure with columns of potentially different types. This is the primary object you will use in Big Data projects.

II. DATA INGESTION: LOADING BIG DATA

Before you can clean data, you must load it from various sources. Pandas supports a wide range of formats used in industrial pipelines:

- **CSV/TSV:** `pd.read_csv()` — The most common format for structured data.
- **Excel:** `pd.read_excel()` — Often used for business reports.
- **JSON:** `pd.read_json()` — Standard for web APIs and NoSQL exports.
- **SQL:** `pd.read_sql()` — Connecting directly to databases like MySQL or PostgreSQL.
- **Parquet:** `pd.read_parquet()` — A columnar storage format optimized for Big Data ecosystems (Hadoop/Spark).

III. DATA EXPLORATION AND INSPECTION

Once loaded, a Data Scientist must "sanity check" the data. Essential commands include:

- `df.head()` / `df.tail()`: Viewing the first/last rows.
- `df.info()`: Checking data types and memory usage.
- `df.describe()`: Generating descriptive statistics (mean, std, min, max) for numerical columns.
- `df.shape`: Understanding the dimensions (Number of records vs. Number of features).

IV. HANDLING MISSING DATA (THE "DIRTY DATA" PROBLEM)

In real-world Big Data, datasets are rarely complete. Missing values (represented as NaN or None) can break Machine Learning algorithms.

4.1. Detection

- `df.isnull().sum()`: Identifying which columns have holes and how many.

4.2. Strategies for Treatment

1. **Deletion (dropna):** Removing rows or columns with missing values. Use this only if the amount of missing data is negligible (< 5%).
2. **Imputation (fillna):** Filling holes with a specific value.
 - **Constant:** Filling with 0 or "Unknown".

- **Statistical:** Filling with the **Mean**, **Median**, or **Mode** of the column.
- **Forward/Backward Fill:** Useful for time-series data where the next value is likely similar to the previous one.

V. DATA TRANSFORMATION AND STANDARDIZATION

5.1. Data Type Conversion

Often, numbers are loaded as "objects" (strings). Using `pd.to_numeric()` or `df.astype()` is required to enable mathematical calculations.

5.2. Handling Duplicates

Big Data pipelines often ingest the same record multiple times.

- `df.duplicated()`: Finding clones.
- `df.drop_duplicates()`: Keeping only the first occurrence to ensure statistical integrity.

5.3. Renaming and Dropping Columns

- `df.rename()`: Making column names "Pythonic" (e.g., lowercase, no spaces).
- `df.drop()`: Removing irrelevant features to reduce memory footprint and prevent "Overfitting" in ML models.

VI. PRACTICAL LAB: THE CLEANING PIPELINE

Python

```
import pandas as pd
```

```
# 1. Load the dataset
```

```
df = pd.read_csv("dtu_student_records.csv")
```

```
# 2. Basic Cleaning
```

```
df.columns = [c.lower().replace(" ", "_") for c in df.columns]
```

```
df.drop_duplicates(inplace=True)
```

```
# 3. Filling missing grades with the median
```

```
median_grade = df['final_score'].median()
```

```
df['final_score'] = df['final_score'].fillna(median_grade)
```

4. Filtering relevant records

```
graduated_students = df[df['status'] == 'Graduated']
```